Source Code

Final Arduino firmware — with inline comments for clarity.

SmartFire_Rover.Full_Script.code:

```
// ============================================================
//  SmartFire Rover — Full Arduino Script
//  Features: Fire Detection, Navigation, Suppression,
//            GPS Location Tracking + Email Alert via ESP8266
//
//  Hardware:
//    - Arduino UNO R3
//    - 3x IR Flame Sensors (Left, Center, Right)
//    - HC-SR04 Ultrasonic Sensor
//    - L298N Motor Driver
//    - SG90 Servo Motor
//    - 12V Mini Water Pump + Relay
//    - LM2596 Voltage Regulator
//    - NEO-6M GPS Module          ← Unique Feature 3
//    - ESP8266 WiFi Module        ← For Email Alert
//
//  Libraries Required (install from Arduino IDE Library Manager):
//    - TinyGPS++ by Mikalhart
//    - SoftwareSerial (built-in)
//    - Servo (built-in)
// ============================================================

// ---------- Library Includes ----------
#include <DHT.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Servo.h>
#include <SoftwareSerial.h>
#include <TinyGPSPlus.h>

// ============================================================
//  PIN DEFINITIONS
// ============================================================

// --- Flame Sensors ---
#define FLAME_LEFT    A0   // Left IR flame sensor
#define FLAME_CENTER  A1   // Center IR flame sensor
#define FLAME_RIGHT   A2   // Right IR flame sensor
#define FLAME_THRESHOLD 500 // Analog threshold (lower = fire detected)
```

```
// --- Ultrasonic Sensor (HC-SR04) ---
#define TRIG_PIN  7
#define ECHO_PIN  6
#define SAFE_DISTANCE 20  // cm — stop if obstacle closer than this


// --- L298N Motor Driver ---
#define IN1  8   // Left motor forward
#define IN2  9   // Left motor backward
#define IN3  10  // Right motor forward
#define IN4  11  // Right motor backward
#define ENA  5   // Left motor speed (PWM)
#define ENB  3   // Right motor speed (PWM)
#define MOTOR_SPEED     180  // Normal speed (0–255)
#define TURN_SPEED      150  // Turning speed

// --- Servo Motor (Nozzle) ---
#define SERVO_PIN  2
#define SERVO_CENTER  90
#define SERVO_LEFT    45
#define SERVO_RIGHT   135

// --- Water Pump Relay ---
#define RELAY_PIN  12
#define PUMP_ON  LOW   // Relay is active LOW
#define PUMP_OFF HIGH

// --- GPS Module (NEO-6M) via SoftwareSerial ---
#define GPS_RX  A3   // Connect to GPS TX
#define GPS_TX  A4   // Connect to GPS RX

// --- ESP8266 WiFi Module via SoftwareSerial ---
#define ESP_RX  A5   // Connect to ESP8266 TX
#define ESP_TX  13   // Connect to ESP8266 RX

// ============================================================
//  WIFI & EMAIL CONFIGURATION
//  ⚠️ Change these to your actual credentials
// ============================================================
const char* WIFI_SSID     = "YOUR_WIFI_NAME";
const char* WIFI_PASSWORD = "YOUR_WIFI_PASSWORD";

// SMTP2GO free SMTP server settings
const char* SMTP_HOST     = "mail.smtp2go.com";
const int   SMTP_PORT     = 2525;
const char* SMTP_USER     = "YOUR_SMTP2GO_USERNAME";   // base64 encoded
const char* SMTP_PASS     = "YOUR_SMTP2GO_PASSWORD";   // base64 encoded
const char* EMAIL_FROM    = "smartfirerover@yourdomain.com";
const char* EMAIL_TO      = "your_email@gmail.com";

// ============================================================
```

```
//  OBJECT DECLARATIONS
// ============================================================
DHT dht(DHT_PIN, DHT_TYPE);
LiquidCrystal_I2C lcd(0x27, 16, 2);   // I2C address 0x27 (try 0x3F if not found)
Servo nozzleServo;
SoftwareSerial gpsSerial(GPS_RX, GPS_TX);
SoftwareSerial espSerial(ESP_RX, ESP_TX);
TinyGPSPlus gps;

// ============================================================
//  STATE VARIABLES
// ============================================================
bool fireDetected      = false;
bool emailSent         = false;   // Send email only once per fire event
float currentLat       = 0.0;
float currentLng       = 0.0;
String gpsLocationStr  = "Location unavailable";

// ============================================================
//  SETUP
// ============================================================
void setup() {
  Serial.begin(9600);

  // Motor pins
  pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT);
  pinMode(ENA, OUTPUT); pinMode(ENB, OUTPUT);

  // Sensor & relay pins
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(RELAY_PIN, OUTPUT);
  digitalWrite(RELAY_PIN, PUMP_OFF);

  // Servo
  nozzleServo.attach(SERVO_PIN);
  nozzleServo.write(SERVO_CENTER);

  // DHT
  dht.begin();

  // LCD
  lcd.init();
  lcd.backlight();
  lcd.setCursor(0, 0);
  lcd.print("SmartFire Rover");
  lcd.setCursor(0, 1);
  lcd.print("Initializing...");
  delay(2000);
  lcd.clear();
```

```
  // GPS Serial
  gpsSerial.begin(9600);

  // ESP8266 Serial
  espSerial.begin(115200);
  delay(1000);

  // Connect WiFi
  connectWiFi();

  lcd.setCursor(0, 0);
  lcd.print("System Ready!");
  lcd.setCursor(0, 1);
  lcd.print("Scanning...");
  delay(1500);
  lcd.clear();
}

// ============================================================
//  MAIN LOOP
// ============================================================
void loop() {

  // 1. Read GPS data continuously
  updateGPS();

  // 2. Read all sensors
  int flameL   = analogRead(FLAME_LEFT);
  int flameC   = analogRead(FLAME_CENTER);
  int flameR   = analogRead(FLAME_RIGHT);
  float temp   = dht.readTemperature();
  long distance = getDistance();

  // 3. Dual-sensor fire validation
  //    Fire confirmed ONLY if flame sensor AND temperature both triggered
  bool flameSeen = (flameL < FLAME_THRESHOLD) ||
                   (flameC < FLAME_THRESHOLD) ||
                   (flameR < FLAME_THRESHOLD);

  bool heatConfirmed = (!isnan(temp) && temp > TEMP_THRESHOLD);

  fireDetected = flameSeen && heatConfirmed;

  // 4. Debug to Serial Monitor
  Serial.print("FL:"); Serial.print(flameL);
  Serial.print(" FC:"); Serial.print(flameC);
  Serial.print(" FR:"); Serial.print(flameR);
  Serial.print(" T:"); Serial.print(temp);
  Serial.print(" D:"); Serial.println(distance);
```

```
  // 5. Act based on fire status
  if (fireDetected) {
    handleFireDetected(flameL, flameC, flameR, distance, temp);
  } else {
    handleNoFire();
    emailSent = false;  // Reset so next fire event sends a new email
  }

  delay(100);
}

// ============================================================
//  FIRE HANDLING
// ============================================================
void handleFireDetected(int fL, int fC, int fR, long dist, float temp) {

  // Show on LCD
  lcd.setCursor(0, 0);
  lcd.print("FIRE DETECTED!  ");
  lcd.setCursor(0, 1);
  lcd.print("T:");
  lcd.print((int)temp);
  lcd.print("C D:");
  lcd.print(dist);
  lcd.print("cm  ");

  // Send GPS email alert ONCE per fire event
  if (!emailSent) {
    sendFireAlert(temp);
    emailSent = true;
  }

  // If obstacle too close — stop and pump
  if (dist < SAFE_DISTANCE) {
    stopMotors();
    activatePump(fL, fC, fR);
    return;
  }

  // Navigate toward fire
  if (fC < FLAME_THRESHOLD) {
    // Fire straight ahead — move forward
    moveForward();
    nozzleServo.write(SERVO_CENTER);
  } else if (fL < FLAME_THRESHOLD && fR >= FLAME_THRESHOLD) {
    // Fire on the left — turn left
    turnLeft();
    nozzleServo.write(SERVO_LEFT);
  } else if (fR < FLAME_THRESHOLD && fL >= FLAME_THRESHOLD) {
    // Fire on the right — turn right
    turnRight();
```

```
    nozzleServo.write(SERVO_RIGHT);
  } else {
    // Both sides — move forward slowly
    moveForwardSlow();
  }
}

// ============================================================
//  PUMP CONTROL
// ============================================================
void activatePump(int fL, int fC, int fR) {
  lcd.setCursor(0, 0);
  lcd.print("Pump ON!        ");
  lcd.setCursor(0, 1);
  lcd.print("Suppressing...  ");

  digitalWrite(RELAY_PIN, PUMP_ON);

  // Sweep nozzle while pumping
  if (fL < FLAME_THRESHOLD) {
    nozzleServo.write(SERVO_LEFT);
    delay(800);
  }
  if (fC < FLAME_THRESHOLD) {
    nozzleServo.write(SERVO_CENTER);
    delay(800);
  }
  if (fR < FLAME_THRESHOLD) {
    nozzleServo.write(SERVO_RIGHT);
    delay(800);
  }

  // Run pump for 3 seconds per cycle
  delay(3000);
  digitalWrite(RELAY_PIN, PUMP_OFF);
  nozzleServo.write(SERVO_CENTER);

  lcd.setCursor(0, 0);
  lcd.print("Pump OFF        ");
  lcd.setCursor(0, 1);
  lcd.print("Checking...     ");
  delay(1000);
}

// ============================================================
//  NO FIRE STATE
// ============================================================
void handleNoFire() {
  stopMotors();
  digitalWrite(RELAY_PIN, PUMP_OFF);
  nozzleServo.write(SERVO_CENTER);
```

```
  lcd.setCursor(0, 0);
  lcd.print("Scanning...     ");
  lcd.setCursor(0, 1);

  float temp = dht.readTemperature();
  lcd.print("Temp:");
  lcd.print(isnan(temp) ? 0 : (int)temp);
  lcd.print("C Safe  ");
}

// ============================================================
//  GPS — Read and update location
// ============================================================
void updateGPS() {
  unsigned long start = millis();
  while (millis() - start < 50) {   // Read GPS for 50ms per loop
    if (gpsSerial.available()) {
      gps.encode(gpsSerial.read());
    }
  }

  if (gps.location.isValid() && gps.location.isUpdated()) {
    currentLat = gps.location.lat();
    currentLng = gps.location.lng();

    // Format: "23.810331, 90.412521"
    gpsLocationStr  = String(currentLat, 6);
    gpsLocationStr += ", ";
    gpsLocationStr += String(currentLng, 6);

    Serial.print("GPS: ");
    Serial.println(gpsLocationStr);
  }
}

// ============================================================
//  WIFI — Connect ESP8266 to WiFi
// ============================================================
void connectWiFi() {
  lcd.setCursor(0, 0);
  lcd.print("Connecting WiFi ");
  lcd.setCursor(0, 1);
  lcd.print(WIFI_SSID);

  sendATCommand("AT+RST", 2000);              // Reset ESP8266
  sendATCommand("AT+CWMODE=1", 1000);         // Station mode
  sendATCommand("AT+CWJAP=\"" + String(WIFI_SSID) +
                "\",\"" + String(WIFI_PASSWORD) + "\"", 8000);

  lcd.clear();
```

```
  lcd.setCursor(0, 0);
  lcd.print("WiFi Connected! ");
  delay(1500);
  lcd.clear();
}

// ============================================================
//  EMAIL — Send fire alert with GPS location
// ============================================================
void sendFireAlert(float temperature) {

  lcd.setCursor(0, 0);
  lcd.print("Sending Alert...");
  lcd.setCursor(0, 1);
  lcd.print("via GPS+Email   ");

  // Build email body
  String body  = "FIRE ALERT from SmartFire Rover!\r\n\r\n";
  body += "A fire has been detected. Immediate action required.\r\n\r\n";
  body += "--- FIRE DETAILS ---\r\n";
  body += "Temperature at detection: " + String(temperature, 1) + " deg C\r\n";
  body += "GPS Location: " + gpsLocationStr + "\r\n";
  body += "Google Maps: https://maps.google.com/?q=" + gpsLocationStr + "\r\n\r\n";
  body += "--- RECOMMENDED ACTION ---\r\n";
  body += "Please contact your local Fire Service immediately.\r\n";
  body += "Bangladesh Fire Service: 199 / 01730-336699\r\n\r\n";
  body += "-- Sent automatically by SmartFire Rover --\r\n";

  // Connect to SMTP server
  String connectCmd = "AT+CIPSTART=\"TCP\",\"";
  connectCmd += SMTP_HOST;
  connectCmd += "\",";
  connectCmd += SMTP_PORT;
  sendATCommand(connectCmd, 5000);
  delay(500);

  // Send SMTP handshake
  smtpSend("EHLO smartfirerover\r\n");
  delay(500);
  smtpSend("AUTH LOGIN\r\n");
  delay(500);
  smtpSend(String(SMTP_USER) + "\r\n");  // base64 username
  delay(500);
  smtpSend(String(SMTP_PASS) + "\r\n");  // base64 password
  delay(500);
  smtpSend("MAIL FROM:<" + String(EMAIL_FROM) + ">\r\n");
  delay(500);
  smtpSend("RCPT TO:<" + String(EMAIL_TO) + ">\r\n");
  delay(500);
  smtpSend("DATA\r\n");
  delay(500);
```

```
  // Email headers + body
  String email  = "From: SmartFire Rover <" + String(EMAIL_FROM) + ">\r\n";
  email += "To: " + String(EMAIL_TO) + "\r\n";
  email += "Subject: FIRE ALERT — SmartFire Rover Detected a Fire!\r\n";
  email += "Content-Type: text/plain; charset=UTF-8\r\n";
  email += "\r\n";
  email += body;
  email += "\r\n.\r\n";   // End of DATA
  smtpSend(email);
  delay(1000);

  smtpSend("QUIT\r\n");
  delay(500);

  sendATCommand("AT+CIPCLOSE", 1000);

  lcd.setCursor(0, 0);
  lcd.print("Alert Sent!     ");
  lcd.setCursor(0, 1);
  lcd.print("Email OK        ");
  delay(2000);
  lcd.clear();

  Serial.println("Email alert sent successfully!");
}

// ============================================================
//  HELPER — Send AT command to ESP8266
// ============================================================
void sendATCommand(String cmd, int timeout) {
  espSerial.println(cmd);
  long t = millis();
  while (millis() - t < timeout) {
    if (espSerial.available()) {
      Serial.write(espSerial.read());
    }
  }
}

// ============================================================
//  HELPER — Send SMTP data via ESP8266 CIPSEND
// ============================================================
void smtpSend(String data) {
  String cmd = "AT+CIPSEND=" + String(data.length());
  espSerial.println(cmd);
  delay(300);
  espSerial.print(data);
  delay(500);
  while (espSerial.available()) {
    Serial.write(espSerial.read());
```

```
  }
}

// ============================================================
//  ULTRASONIC — Get distance in cm
// ============================================================
long getDistance() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  long duration = pulseIn(ECHO_PIN, HIGH, 30000);  // 30ms timeout
  if (duration == 0) return 999;                   // No echo = far away
  return duration * 0.034 / 2;
}

// ============================================================
//  MOTOR CONTROL FUNCTIONS
// ============================================================
void moveForward() {
  analogWrite(ENA, MOTOR_SPEED);
  analogWrite(ENB, MOTOR_SPEED);
  digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
  digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);
}

void moveForwardSlow() {
  analogWrite(ENA, 120);
  analogWrite(ENB, 120);
  digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
  digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);
}

void turnLeft() {
  analogWrite(ENA, TURN_SPEED);
  analogWrite(ENB, TURN_SPEED);
  digitalWrite(IN1, LOW);  digitalWrite(IN2, HIGH);  // Left motor backward
  digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);   // Right motor forward
}

void turnRight() {
  analogWrite(ENA, TURN_SPEED);
  analogWrite(ENB, TURN_SPEED);
  digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);   // Left motor forward
  digitalWrite(IN3, LOW);  digitalWrite(IN4, HIGH);  // Right motor backward
}

void stopMotors() {
  analogWrite(ENA, 0);
```

```
  analogWrite(ENB, 0);
  digitalWrite(IN1, LOW); digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW); digitalWrite(IN4, LOW);
}
```

Email_script.code:

```
#include <WiFiManager.h>
#include <ESP_Mail_Client.h>
#include <TinyGPS++.h>
#include <SoftwareSerial.h>

static const int RXPin = D5, TXPin = D6;
static const uint32_t GPSBaud = 9600;

// The TinyGPS++ object
TinyGPSPlus gps;

// The serial connection to the GPS device
SoftwareSerial ss(RXPin, TXPin);

/** The smtp host name e.g. smtp.gmail.com for GMail or smtp.office365.com for Outlook or smtp.mail.yahoo.com */
#define SMTP_HOST "smtp.gmail.com"
#define SMTP_PORT 465

/* The sign in credentials */
#define AUTHOR_EMAIL "esp32.hat@gmail.com"
#define AUTHOR_PASSWORD "kwxlrfbrbfhhthum"

/* Recipient's email*/
#define RECIPIENT_EMAIL "23101160@uap-bd.edu"

SMTPSession smtp;

void smtpCallback(SMTP_Status status);

int last = 0;

char* esp_ssid = "IoT Project";
char* esp_pass = "1234567890";

const int Erasing_button = 0;
```

```
#define fire_pin    D0

float latitude = 23.754874956773744;
float longitude = 90.38947913767419;

void configModeCallback (WiFiManager *myWiFiManager) {
  Serial.println(WiFi.softAPIP());
  Serial.println(myWiFiManager->getConfigPortalSSID());
}

void setup() {
  Serial.begin(9600);
  ss.begin(GPSBaud);
  pinMode(fire_pin, INPUT);
  pinMode(Erasing_button, INPUT);
  pinMode(D4, OUTPUT);

  for (uint8_t t = 4; t > 0; t--) {
    digitalWrite(D4, LOW);
    delay(500);
    Serial.println(t);
    digitalWrite(D4, HIGH);
    delay(500);
  }

  // Press and hold the button to erase all the credentials
  if (digitalRead(Erasing_button) == LOW)
  {
    WiFiManager wifiManager;
    wifiManager.resetSettings();
    ESP.restart();
    delay(1000);
  }

  WiFiManager wifiManager;
  wifiManager.setAPCallback(configModeCallback);
  if (!wifiManager.autoConnect(esp_ssid, esp_pass)) {
    Serial.println("failed to connect and hit timeout");
    ESP.restart();
    delay(1000);
  }
  Serial.println("connected...yeey :)");
  digitalWrite(D4, LOW);
}

void loop() {
```

```
  int fast = digitalRead(fire_pin);
  if (fast != last) {
    if (fast == HIGH) {
      String SMS = "http://maps.google.com/maps?q=loc:" + String(latitude, 8) + "," +
String(longitude, 8);
      mail(SMS);
    }
  }
  last = fast;

  unsigned long start = millis();
  do {
    while (ss.available())
      if (gps.encode(ss.read()))
        displayInfo();
  } while (millis() - start < 100);
}

void mail(String data_send) {
  MailClient.networkReconnect(true);
  smtp.debug(1);

  /* Set the callback function to get the sending results */
  smtp.callback(smtpCallback);

  /* Declare the Session_Config for user defined session credentials */
  Session_Config config;

  /* Set the session config */
  config.server.host_name = SMTP_HOST;
  config.server.port = SMTP_PORT;
  config.login.email = AUTHOR_EMAIL;
  config.login.password = AUTHOR_PASSWORD;
  config.login.user_domain = "";

  config.time.ntp_server = F("pool.ntp.org,time.nist.gov");
  config.time.gmt_offset = 3;
  config.time.day_light_offset = 0;

  /* Declare the message class */
  SMTP_Message message;

  /* Set the message headers */
  message.sender.name = F("Fire Alert");
  message.sender.email = AUTHOR_EMAIL;
  message.subject = F("Fire Alert");
```

```
  message.addRecipient(F("Owner"), RECIPIENT_EMAIL);

  /*Send HTML message*/
  String htmlMsg = "<div style=\"color:#2f4468;\"><h1>Fire Alert Status.....</h1><p>" +
data_send + "</p></div>";
  message.html.content = htmlMsg.c_str();
  message.html.content = htmlMsg.c_str();
  message.text.charSet = "us-ascii";
  message.html.transfer_encoding = Content_Transfer_Encoding::enc_7bit;

  message.priority = esp_mail_smtp_priority::esp_mail_smtp_priority_low;
  message.response.notify = esp_mail_smtp_notify_success | esp_mail_smtp_notify_failure |
esp_mail_smtp_notify_delay;

  /* Connect to the server */
  if (!smtp.connect(&config)) {
    ESP_MAIL_PRINTF("Connection error, Status Code: %d, Error Code: %d, Reason: %s",
smtp.statusCode(), smtp.errorCode(), smtp.errorReason().c_str());
    return;
  }

  if (!smtp.isLoggedIn()) {
    Serial.println("\nNot yet logged in.");
  }
  else {
    if (smtp.isAuthenticated())
      Serial.println("\nSuccessfully logged in.");
    else
      Serial.println("\nConnected with no Auth.");
  }

  /* Start sending Email and close the session */
  if (!MailClient.sendMail(&smtp, &message))
    ESP_MAIL_PRINTF("Error, Status Code: %d, Error Code: %d, Reason: %s",
smtp.statusCode(), smtp.errorCode(), smtp.errorReason().c_str());
}

/* Callback function to get the Email sending status */
void smtpCallback(SMTP_Status status) {
  /* Print the current status */
  Serial.println(status.info());

  /* Print the sending result */
  if (status.success()) {
    // ESP_MAIL_PRINTF used in the examples is for format printing via debug Serial port
    // that works for all supported Arduino platform SDKs e.g. AVR, SAMD, ESP32 and
```

```
ESP8266.
   // In ESP8266 and ESP32, you can use Serial.printf directly.

   Serial.println("----------------");
   ESP_MAIL_PRINTF("Message sent success: %d\n", status.completedCount());
   ESP_MAIL_PRINTF("Message sent failed: %d\n", status.failedCount());
   Serial.println("----------------\n");

   for (size_t i = 0; i < smtp.sendingResult.size(); i++)
   {
    /* Get the result item */
    SMTP_Result result = smtp.sendingResult.getItem(i);

    // In case, ESP32, ESP8266 and SAMD device, the timestamp get from result.timestamp
should be valid if
    // your device time was synched with NTP server.
    // Other devices may show invalid timestamp as the device time was not set i.e. it will show
Jan 1, 1970.
    // You can call smtp.setSystemTime(xxx) to set device time manually. Where xxx is
timestamp (seconds since Jan 1, 1970)

    ESP_MAIL_PRINTF("Message No: %d\n", i + 1);
    ESP_MAIL_PRINTF("Status: %s\n", result.completed ? "success" : "failed");
    ESP_MAIL_PRINTF("Date/Time: %s\n",
MailClient.Time.getDateTimeString(result.timestamp, "%B %d, %Y %H:%M:%S").c_str());
    ESP_MAIL_PRINTF("Recipient: %s\n", result.recipients.c_str());
    ESP_MAIL_PRINTF("Subject: %s\n", result.subject.c_str());
   }
   Serial.println("----------------\n");

   // You need to clear sending result as the memory usage will grow up.
   smtp.sendingResult.clear();
  }
}

void displayInfo() {
  if (gps.location.isValid()) {
   latitude = (gps.location.lat());  //Storing the Lat. and Lon.
   longitude = (gps.location.lng());

   Serial.print("LAT:  ");
   Serial.println(latitude, 8);  // float to x decimal places
   Serial.print("LONG: ");
   Serial.println(longitude, 8);
  }
}
```